DeployGuard

17 modules · working explanations · live demos · 85 viva questions

Use this guide

Study each module in the same order: how it works, where it is implemented, how to demonstrate it, and five questions with complete answers.

Evidence boundary

This document formats the supplied repository-based notes. The conversion does not independently run the code. Its final section identifies live AWS and external behavior that remains unverified.

Modules

1. User Authentication and Audit Logging
2. Project and Workspace Management
3. Rule-Based Stack Detection Engine
4. DevOps Template Engine and Pre-Flight Validation
5. CI/CD Pipeline and Asynchronous Queuing
6. Security Scanning and Policy Enforcement
7. Predictive Cost Analysis
8. Infrastructure Provisioning and Service Discovery
9. Distributed State Management
10. Persistent Storage and Backup Recovery
11. ECS Orchestration and Rollback
12. Monitoring and Observability
13. AI Troubleshooting
14. Alerting and Notifications
15. Infrastructure Lifecycle and Destroy
16. Terraform Export
17. Subscription and Billing

1. User Authentication and Audit Logging

How it works

The user starts GitHub OAuth. DeployGuard creates an OAuth state value, stores it in a protected cookie, and sends the user to GitHub. After GitHub returns, the backend validates the state and authorization code. It then creates or loads the user and stores a session token in an HTTP-only cookie.

For every protected API request, middleware reads the session and attaches the user to the request. Role guards then check whether the user is an administrator, developer, or read-only user. Important actions are saved in the audit log with the user, action, resource, status, IP address, and sanitized metadata.

Main files

- backend/src/auth/auth.controller.ts
- backend/src/auth/auth.service.ts
- backend/src/common/middleware/authenticated-user.middleware.ts
- backend/src/common/rbac/require-role.guard.ts
- backend/src/audit-log/audit-log.service.ts
- frontend/src/context/AuthContext.jsx

Live demonstration

1. Log in with GitHub.
2. Open a protected project page.
3. Log in as a read-only user.
4. Try to deploy or update project settings.
5. Show the permission error.
6. Open the audit-log page.
7. Show login, logout, and denied-action records.

Five viva questions

Question 1: How does authentication work?

The user is redirected to GitHub through OAuth. DeployGuard validates the returned OAuth state and code, loads the user, and creates a session cookie. Later requests use that session cookie to identify the authenticated user.

Question 2: How does role-based access work?

The authenticated user has a role such as administrator, developer, or read-only. The require-role.guard.ts file checks the user's role before allowing the controller method to execute. If the user is not allowed, the backend returns an authorization error.

Question 3: Where is the session stored?

The session is sent as an HTTP-only cookie. The browser sends it automatically with requests, but frontend JavaScript cannot directly read it. The backend uses the cookie to find the user.

Question 4: What is recorded in the audit log?

The audit log records the actor, action, resource type, resource ID, status, IP address, user agent, and sanitized metadata. Sensitive values such as passwords, tokens, cookies, and secrets are masked.

Question 5: How will you demonstrate this module?

I will log in, open a protected project, attempt an action with a restricted role, and show that the backend rejects it. Then I will open the audit page and show the authentication and authorization events.

2. Project and Workspace Management

How it works

A user creates a project and connects a GitHub repository. The project stores the repository URL, repository name, branch, environment, deployment directories, services, environment variables, and deployment history.

The backend exposes project APIs. The frontend uses these APIs to show project cards, project details, settings, pipeline history, infrastructure, and current state.

Main files

- backend/src/projects/projects.controller.ts
- backend/src/projects/projects.service.ts
- backend/src/projects/github-app.service.ts
- backend/src/projects/repository-source.service.ts
- backend/src/projects/current-state/project-current-state.service.ts
- frontend/src/pages/Projects.jsx
- frontend/src/pages/NewProject.jsx
- frontend/src/pages/ProjectDetails.jsx

Live demonstration

1. Open New Project.
2. Select a GitHub repository.
3. Select a branch.
4. Select a deployment directory.
5. Save the project.
6. Open the project overview.
7. Show project status and deployment history.

Five viva questions

Question 1: What information is stored for a project?

The project stores the repository, branch, environment, deployment directory, services, environment configuration, database configuration, deployment operations, and current deployment state.

Question 2: How does GitHub repository selection work?

The backend uses the GitHub App integration to retrieve repositories available to the user. The selected repository is saved in the project and is used later when DeployGuard dispatches the deployment workflow.

Question 3: How does the workspace show the current status?

The backend creates a current-state projection from deployment operations, pipeline stages, stable releases, and failure evidence. The frontend reads this projection and displays states such as configured, deploying, live, failed, or destroyed.

Question 4: How are deployment histories stored?

Each deployment is represented by a pipeline operation. The operation stores the source commit, status, stages, timestamps, workflow information, failure information, and release evidence.

Question 5: How will you demonstrate this module?

I will create a project, select a repository and branch, save it, and open the project overview. Then I will open the Pipeline page to show the deployment history and the current state.

3. Rule-Based Stack Detection Engine

How it works

DeployGuard examines repository files to identify the application structure. It checks files such as package manifests, lockfiles, workspace configuration, and service directories.

It determines the package manager, workspace root, service directory, build root, install root, package identity, build command, start command, and service port. The result becomes a canonical build target connected to the source SHA.

Main files

- backend/src/projects/build-target-resolver.service.ts
- backend/src/projects/build-target.ts
- backend/src/projects/service-port-resolution.ts
- backend/scripts/verify-build-target-resolution.ts
- backend/scripts/verify-service-port-resolution.ts

Live demonstration

1. Select a repository.
2. Show the detected package manager.
3. Show the selected service directory.
4. Show the resolved build target.
5. Show the detected service port.
6. Try an unsupported or ambiguous repository.

Five viva questions

Question 1: What does the detection engine detect?

It detects the repository structure, package manager, workspace membership, service directory, build scope, start command, and application port.

Question 2: How does it detect a workspace application?

It checks workspace files and package manifests. If the selected service belongs to a workspace, it keeps the correct workspace root and creates the correct package-specific build and start commands.

Question 3: What happens if the repository is unsupported?

DeployGuard does not guess. It returns a structured deployment-contract error and stops before building or provisioning infrastructure.

Question 4: Why is the detected port important?

The port is used during local container validation, ECS task definition generation, load-balancer configuration, and runtime verification. If the port is wrong, the application may start but remain unreachable.

Question 5: How will you demonstrate this module?

I will show the repository's detected build target and port in the project or pipeline evidence. Then I will use an unsupported fixture and show that DeployGuard blocks deployment instead of selecting an unsafe default.

4. DevOps Template Engine and Pre-Flight Validation

How it works

DeployGuard creates a deployment contract before AWS changes. The contract contains services, source SHA, build target, port, runtime variables, secrets, and database information.

The workflow builds the image locally and runs it in a validation environment. It checks that the process remains alive, listens on the expected port, and binds to 0.0.0.0.

Main files

- backend/src/projects/deployment-requirement-resolver.service.ts
- backend/src/projects/pipeline/docker-build.service.ts
- backend/src/projects/security/dockerfile-security.service.ts
- .github/workflows/deployguard-reusable.yml
- backend/scripts/verify-deployment-requirements.ts

Live demonstration

1. Remove a required variable.
2. Start deployment.
3. Show that admission blocks it.
4. Use an application with an incorrect port.
5. Show the runtime validation failure.

Five viva questions

Question 1: What is pre-flight validation?

Pre-flight validation checks whether the application is deployable before AWS infrastructure is changed. It validates required configuration, runtime values, secrets, ports, and container startup.

Question 2: How does container validation work?

The workflow builds the image and starts it locally. It checks the container's running state and tests the required port from a separate network peer. This prevents a localhost-only process from appearing healthy.

Question 3: What happens if a required variable is missing?

The deployment is stopped during admission. The user receives a structured validation message instead of reaching Terraform with incomplete runtime configuration.

Question 4: What failure does a wrong bind address cause?

If the application listens only on 127.0.0.1, it may be alive inside the container but unreachable from ECS or the load balancer. DeployGuard reports an external binding or reachability failure.

Question 5: How will you demonstrate this module?

I will intentionally remove a required setting or use an application that listens on the wrong port. Then I will show the failed pre-flight stage and the structured failure evidence.

5. CI/CD Pipeline and Asynchronous Queuing

How it works

When the user clicks Deploy, the backend creates an operation and dispatches GitHub Actions. The long-running work is performed by the reusable workflow.

DeployGuard later reads workflow runs, jobs, logs, stages, artifacts, and final evidence. The frontend polls the backend and displays the operation progress.

Main files

- backend/src/projects/pipeline/pipeline.queue.ts
- backend/src/projects/pipeline/github-actions.service.ts
- backend/src/projects/pipeline/pipeline-activity.service.ts
- backend/src/projects/railpack-deployment.service.ts
- .github/workflows/deployguard-reusable.yml
- frontend/src/components/projects/PipelineExecution.jsx

Live demonstration

1. Click Deploy.
2. Show the operation ID.
3. Open the Pipeline page.
4. Show stage progress.
5. Open workflow logs.
6. Show the final result.

Five viva questions

Question 1: What happens immediately after clicking Deploy?

The backend validates the request, creates a deployment operation, and dispatches the workflow. The API returns operation information instead of waiting for the entire deployment.

Question 2: What does GitHub Actions do?

It checks out the exact source, builds and validates the image, pushes it to ECR, scans it, prepares Terraform, applies infrastructure, verifies ECS, and uploads evidence.

Question 3: How does the frontend know the deployment status?

The frontend calls backend status and current-state APIs. The backend reconciles GitHub Actions information and stores it in PostgreSQL before returning it to the frontend.

Question 4: What happens if GitHub Actions fails?

DeployGuard reads the failed stage and workflow evidence. It stores a deterministic failure code, stage, owner, and safe evidence, then shows the result in the Pipeline page.

Question 5: How will you demonstrate this module?

I will click Deploy, show the operation ID, refresh the Pipeline page, show the individual stages, open the logs, and display the final success or failure evidence.

6. Security Scanning and Policy Enforcement

How it works

After the image is built and pushed to ECR, Trivy scans the exact immutable image. The workflow collects vulnerability findings, classifies them, hashes the evidence, and applies the security policy.

Advisory mode records findings. Enforced mode can block a deployment when the policy condition is met.

Main files

- .github/workflows/deployguard-reusable.yml
- backend/scripts/verify-trivy-controls.ts
- backend/src/projects/security
- Security evidence entities and services

Live demonstration

1. Run a clean scan.
2. Show vulnerability counts.
3. Run advisory mode.
4. Run enforced mode.
5. Show the blocked deployment.

Five viva questions

Question 1: What exactly is scanned?

The exact immutable container image that will be deployed is scanned. This is stronger than scanning only source code because the image contains the real runtime dependencies.

Question 2: What does advisory mode do?

Advisory mode records the scan result and displays the findings, but it does not stop the deployment. It is useful when the team wants visibility before enforcing policy.

Question 3: What does enforced mode do?

Enforced mode evaluates the configured policy. A fixable Critical application dependency vulnerability can cause the workflow to fail and the deployment to be blocked.

Question 4: How is scan evidence protected?

The findings are normalized into structured JSON and a SHA-256 evidence hash is stored. This makes later changes to the scan result detectable.

Question 5: How will you demonstrate this module?

I will run the same fixture first in advisory mode and then in enforced mode. I will show that the first records a warning while the second blocks the release.

7. Predictive Cost Analysis

How it works

DeployGuard generates a Terraform cost plan and can pass it to Infracost. The result is normalized into resource-level estimates and stored with the project or deployment.

The system also supports explicit mock mode if real Infracost is not configured.

Main files

- backend/src/finops/finops.service.ts
- backend/src/finops/infracost.service.ts
- backend/src/finops/terraform-cost-plan.service.ts
- backend/src/finops/project-cost-estimate.entity.ts
- frontend/src/pages/Billing.jsx

Live demonstration

1. Open cost analysis.
2. Generate an estimate.
3. Show the resource breakdown.
4. Show the total monthly cost.
5. Show warning or approval state.

Five viva questions

Question 1: What is the source of the cost estimate?

The cost estimate is based on the planned infrastructure. The real path converts a Terraform plan to JSON and sends it to Infracost for calculation.

Question 2: What resources can appear in the estimate?

The breakdown can include ECS/Fargate, load balancers, EFS, CloudWatch logs, database resources, and backup-related resources.

Question 3: Can cost stop a deployment?

Yes. The cost policy can create a warning, require approval, or block the deployment when the configured limit is exceeded.

Question 4: How can you tell if the estimate is mock data?

The service records that mock mode was used. The pipeline event and estimate metadata should identify whether the result came from real Infracost or a mock calculation.

Question 5: How will you demonstrate this module?

I will generate a cost estimate, show the individual resources and total amount, then show either an approval-required state or a cost policy warning.

8. Infrastructure Provisioning and Service Discovery

How it works

Terraform creates AWS infrastructure from the canonical runtime configuration. DeployGuard then records the infrastructure events, ECS identity, target health, service-discovery information, and public endpoint.

Main files

- infrastructure/railpack-runtime
- backend/terraform/modules
- backend/src/projects/pipeline/terraform.service.ts
- backend/src/infrastructure/service-discovery.service.ts
- backend/src/orchestration/alb.service.ts

Live demonstration

1. Open Infrastructure.
2. Show Terraform plan.
3. Show Terraform apply.
4. Show ECS service.
5. Show ALB target health.
6. Open the endpoint.

Five viva questions

Question 1: What infrastructure does Terraform create?

It can create ECS clusters and services, task definitions, ALB resources, target groups, security groups, IAM roles, EFS, Cloud Map, secrets infrastructure, and CloudWatch log groups.

Question 2: What is service discovery used for?

Service discovery allows services to find each other through stable names instead of hard-coded task IP addresses. This is useful when containers are replaced or scaled.

Question 3: How is the public endpoint recorded?

After infrastructure and runtime verification, DeployGuard stores the endpoint and service-discovery record in the infrastructure and project read models.

Question 4: Does Terraform decide whether the application is healthy?

No. Terraform creates and updates infrastructure. DeployGuard checks ECS stability, target health, runtime evidence, and public reachability.

Question 5: How will you demonstrate this module?

I will show the Terraform stages, then display the created ECS service, ALB target health, service discovery record, and public endpoint.

9. Distributed State Management

How it works

DeployGuard stores operational deployment state in PostgreSQL and Terraform state remotely in S3. The state module manages versions, locks, validation, checksums, snapshots, and recovery.

Main files

- backend/src/state-management/state-management.service.ts
- backend/src/state-management/terraform-state.service.ts
- backend/src/state-management/state-lock.service.ts
- backend/src/state-management/state-recovery.service.ts
- Terraform S3 backend configuration

Live demonstration

1. Open Infrastructure State.
2. Show the remote state bucket and key.
3. Show current version.
4. Show checksum.
5. Show lock status.

6. Run state validation.

Five viva questions

Question 1: What state is stored in PostgreSQL?

PostgreSQL stores deployment operations, pipeline stages, failures, stable releases, runtime identities, infrastructure events, and current-state projections.

Question 2: What state is stored in Terraform's backend?

Terraform's backend stores the infrastructure state file, resource relationships, outputs, versions, and locking information.

Question 3: Why are locks required?

Locks prevent two Terraform operations from modifying the same state at the same time. Without locks, concurrent changes could corrupt the state or create conflicting infrastructure actions.

Question 4: What happens if state is corrupted?

DeployGuard can validate the state, compare checksums, inspect previous versions, and use controlled recovery procedures. Recovery is restricted because restoring the wrong version can damage infrastructure.

Question 5: How will you demonstrate this module?

I will open the state page and show the bucket, state key, version, checksum, lock status, and validation result.

10. Persistent Storage and Backup Recovery

How it works

DeployGuard stores application data outside the temporary application container. Terraform creates EFS, mount targets, access points, and optional AWS Backup resources. ECS tasks mount the EFS path.

DeployGuard also stores backup status and restore requests.

Main files

- backend/src/storage/storage.service.ts
- backend/src/storage/backup.service.ts
- backend/src/storage/efs.service.ts
- backend/src/storage/storage.controller.ts
- backend/terraform/modules/efs
- backend/terraform/modules/database-service

Live demonstration

1. Enable persistent storage.
2. Deploy a disposable application.
3. Show EFS settings.
4. Show backup status.
5. Submit a restore request.
6. Show the restore event.

Five viva questions

Question 1: Why is EFS needed?

Containers are replaceable and their local filesystem should not be treated as permanent storage. EFS provides persistent storage that can survive container replacement.

Question 2: How does an ECS task access EFS?

Terraform creates the filesystem, mount targets, access point, security rules, and IAM permissions. The ECS task definition then mounts the configured EFS volume into the container.

Question 3: How is backup recovery represented?

The backup service records backup status, recovery points, restore requests, and storage events. These records are shown through the storage and infrastructure interfaces.

Question 4: Can Reset and Deploy Fresh delete healthy database data?

It should not. The reset flow uses managed-database reconciliation and recovery decisions. Healthy persistent data is preserved, and a required recovery state must not be silently treated as safe to reset.

Question 5: How will you demonstrate this module?

I will use a disposable project, enable storage, show the EFS configuration, show backup information, and create a restore request. I will not claim that the restore completed unless AWS proof exists.

11. ECS Orchestration and Rollback

How it works

DeployGuard creates or updates ECS services using immutable task definitions. It waits for ECS stability, checks target health, and records the release as stable only after verification.

Rollback selects a previous stable task definition and updates ECS to it without rebuilding the source.

Main files

- backend/src/orchestration/ecs.service.ts
- backend/src/orchestration/orchestration.service.ts
- backend/src/orchestration/alb.service.ts
- backend/src/orchestration/project-stable-release.entity.ts
- backend/scripts/verify-direct-ecs-redeploy.ts
- backend/scripts/verify-direct-ecs-rollback.ts

Live demonstration

1. Deploy version 1.
2. Deploy version 2.
3. Show version 2 as stable.
4. Open rollback candidates.
5. Select version 1.
6. Show ECS using the old task definition.
7. Show successful verification.

Five viva questions

Question 1: What is stored for a stable release?

DeployGuard stores the source SHA, image URI and digest, runtime configuration revision, task-definition identity, deployment operation, and verification evidence.

Question 2: How does a new release reach ECS?

The workflow registers or selects an immutable task definition and calls ECS service update. ECS performs its rolling deployment and DeployGuard waits for service stability.

Question 3: How does rollback select the correct version?

DeployGuard reads the stored stable-release and rollback-target records. It validates the exact task definition, image, runtime configuration, and database compatibility before updating ECS.

Question 4: Does rollback rebuild the application?

No. Rollback reuses the previous immutable image and task definition. This avoids creating a different artifact from the same source.

Question 5: How will you demonstrate this module?

I will deploy two versions, show both release identities, select the first release as the rollback target, and show that ECS moves back without a new source build.

12. Monitoring and Observability

How it works

DeployGuard collects deployment events, ECS runtime information, CloudWatch logs, CloudWatch metrics, Prometheus metrics, and Grafana data. The frontend presents these through project monitoring screens.

Main files

- backend/src/observability/observability.service.ts
- backend/src/observability/observability.controller.ts
- backend/src/observability/cloudwatch-logs.service.ts
- backend/src/observability/cloudwatch-metrics.service.ts
- backend/src/observability/prometheus.service.ts
- backend/src/observability/live-runtime-resolver.service.ts
- frontend/src/pages/ProjectMetrics.jsx
- monitoring/prometheus.yml
- monitoring/grafana

Live demonstration

1. Open Project Metrics.
2. Show deployment timing.
3. Show current runtime identity.
4. Open live logs.
5. Filter log lines.
6. Show metrics or Grafana.

Five viva questions

Question 1: What monitoring data does DeployGuard show?

It shows deployment stages, ECS runtime information, CloudWatch logs, CloudWatch metrics, application events, and Prometheus-based metrics where configured.

Question 2: How are application logs retrieved?

The backend uses CloudWatch log services to retrieve and stream logs. The frontend displays them in a log viewer with filtering and reconnect behavior.

Question 3: What is the purpose of runtime identity?

Runtime identity connects the displayed logs and metrics to the exact project, environment, service, task, release, and configuration revision.

Question 4: What happens if monitoring is unavailable?

The deployment result should not be guessed as healthy. DeployGuard records the observation failure and shows that runtime evidence is unavailable or incomplete.

Question 5: How will you demonstrate this module?

I will open Metrics, show runtime identity, open logs, filter them, and show deployment monitoring data. I will not claim live AWS monitoring certification unless the configured AWS environment is working.

13. AI Troubleshooting

How it works

The AI module collects bounded evidence from a failed deployment. It creates an analysis session, sends the evidence to the configured provider, stores the result, and allows follow-up questions.

The deterministic deployment engine remains authoritative. AI only explains the evidence.

Main files

- backend/src/ai-troubleshooting/ai-troubleshooting.service.ts
- backend/src/ai-troubleshooting/ai-evidence.service.ts
- backend/src/ai-troubleshooting/ai-evidence-preprocessor.service.ts
- backend/src/ai-troubleshooting/ai-provider.adapter.ts
- frontend/src/pages/ProjectTroubleshooting.jsx

Live demonstration

1. Select a failed deployment.
2. Open Troubleshooting.
3. Start analysis.
4. Show the evidence references.
5. Ask a follow-up question.
6. Show the stored analysis session.

Five viva questions

Question 1: What evidence does the AI receive?

It receives bounded deployment evidence such as the failure code, failed stage, sanitized logs, ECS information, Terraform diagnostics, and target-health details.

Question 2: Where is the AI result stored?

The analysis session, messages, result, provider information, and evidence snapshot are stored in the AI troubleshooting entities.

Question 3: Can the AI mark a deployment as successful?

No. The deterministic DeployGuard lifecycle decides whether the deployment is live, failed, blocked, or safe to retry. AI only provides explanation and guidance.

Question 4: What happens if the provider is unavailable?

DeployGuard can return deterministic evidence-only guidance or show that AI analysis is unavailable. It must not pretend that an external AI response was generated.

Question 5: How will you demonstrate this module?

I will open a failed deployment, start analysis, show the evidence used, ask a follow-up question, and show the persisted result and session history.

14. Alerting and Notifications

How it works

The user configures notification preferences. Deployment events are mapped to notification messages. Notifications are saved, deduplicated, retried when necessary, and sent through the configured provider.

Main files

- backend/src/notifications/notifications.service.ts
- backend/src/notifications/notification-dispatcher.service.ts
- backend/src/notifications/sns-notification.adapter.ts
- backend/src/notifications/notification-preference.entity.ts
- backend/src/notifications/notification-delivery.entity.ts
- frontend/src/components/projects/NotificationSettingsPanel.jsx

Live demonstration

1. Open notification settings.
2. Enable failure notifications.
3. Trigger a controlled failed deployment.
4. Show the notification.
5. Show delivery and retry information.

Five viva questions

Question 1: How is a deployment event converted to a notification?

The notification dispatcher receives the event, checks the user’s preferences, creates a notification record, and sends it through the configured provider.

Question 2: How are duplicate notifications prevented?

The delivery and event identifiers are used to identify whether an equivalent notification was already sent. This prevents repeated messages for the same event.

Question 3: What happens when delivery fails?

The failure is recorded and the dispatcher can retry according to the notification policy. The final delivery status remains visible to the platform.

Question 4: What happens if no provider is configured?

DeployGuard should show the provider as unavailable or disabled. It should not claim that a notification was delivered when only the internal notification record was created.

Question 5: How will you demonstrate this module?

I will configure a notification preference, trigger a controlled deployment event, and show the internal notification record and delivery status.

15. Infrastructure Lifecycle and Destroy

How it works

Destroy requires a confirmation phrase. DeployGuard starts Terraform destruction, waits for completion, verifies AWS deletion, cleans non-Terraform resources, and finalizes the control-plane state.

Main files

- backend/src/infrastructure-lifecycle/infrastructure-lifecycle.controller.ts
- backend/src/infrastructure-lifecycle/infrastructure-lifecycle.service.ts
- backend/src/projects/project-deletion.service.ts
- backend/src/projects/destroy-confirmation.ts
- .github/workflows/deployguard-reusable.yml

Live demonstration

Use a disposable project only:

1. Click Destroy.
2. Enter an incorrect phrase.
3. Show rejection.
4. Enter the correct phrase.
5. Start destroy.
6. Show Terraform destruction.
7. Show deletion verification.
8. Show final destroyed state.

Five viva questions

Question 1: How is destroy protected?

The user must provide the required confirmation phrase. The backend checks it before dispatching the destroy workflow.

Question 2: What does Terraform destroy?

Terraform destroys the infrastructure resources owned by the project, such as ECS, ALB, networking, EFS, and related resources according to the project configuration.

Question 3: How does DeployGuard verify deletion?

After Terraform finishes, DeployGuard performs AWS observation and checks whether the expected project resources are gone. It then records deletion evidence.

Question 4: What happens if deletion is incomplete?

The operation is not treated as completely successful. DeployGuard records the incomplete deletion and preserves cleanup information so it can be retried safely.

Question 5: How will you demonstrate this module?

I will use a disposable project, demonstrate the failed confirmation first, then perform a confirmed destroy and show the Terraform, deletion verification, and final project state.

16. Terraform Export

How it works

DeployGuard takes the canonical runtime configuration and generates Terraform files. It packages the files and allows the user to download them.

The export should represent the same services, images, ports, infrastructure settings, and runtime configuration used by DeployGuard.

Main files

- backend/src/terraform-export/terraform-export.service.ts
- backend/src/terraform-export/terraform-export.controller.ts
- frontend/src/pages/ProjectInfrastructure.jsx
- backend/scripts/verify-terraform-export.ts

Live demonstration

1. Open Infrastructure.
2. Click Export Terraform.
3. Download the archive.
4. Open the Terraform files.
5. Show service, image, port, and infrastructure variables.

Five viva questions

Question 1: What does the export contain?

It contains generated Terraform configuration and variables required to represent the project’s infrastructure and runtime configuration.

Question 2: Where does the export configuration come from?

It comes from the canonical project and runtime configuration, not from arbitrary browser values. This helps keep the export consistent with the deployment.

Question 3: Are secrets exported in plain text?

The export must preserve secret references and avoid exposing secret values. The implementation uses runtime secret references rather than treating secret values as normal public configuration.

Question 4: Why is export useful during a demo?

It proves that the infrastructure configuration is visible and portable. The user can download the generated code and inspect it independently.

Question 5: How will you demonstrate this module?

I will download the export archive, open the Terraform files, and show how the project services, image, port, and infrastructure resources are represented.

17. Subscription and Billing

How it works

The billing module stores subscriptions, plans, checkout sessions, invoices, payments, usage counters, and webhook events. The entitlement service determines the user’s effective plan and checks usage before allowing metered actions.

The current implementation has:

- Free
- Pro
- Pro Plus

Main files

- backend/src/billing/billing-plan.ts
- backend/src/billing/billing.service.ts
- backend/src/billing/billing-provider.service.ts
- backend/src/billing/entitlement.service.ts
- backend/src/billing/billing.controller.ts
- frontend/src/pages/Billing.jsx

Live demonstration

1. Open Billing.
2. Show current plan.
3. Show usage limits.
4. Show available plan features.
5. Open checkout or billing portal.
6. Show invoices.
7. Attempt an action after exceeding a limit.

Five viva questions

Question 1: What plans exist in the implementation?

The implementation contains Free, Pro, and Pro Plus plans. The PDF mentions Free and Pro, so Pro Plus is an implementation difference that should be stated honestly.

Question 2: How are plan limits enforced?

The backend checks the effective subscription and usage counters before allowing metered operations such as AI analysis, notifications, Terraform export, or project creation.

Question 3: What is the role of Stripe?

Stripe can handle checkout, billing portal access, subscription lifecycle, invoices, and webhook events. DeployGuard receives and validates the billing events, then updates its local subscription state.

Question 4: What happens when the subscription is unpaid?

The entitlement service evaluates the subscription status. If the status does not qualify for paid access, the user is resolved to the appropriate lower entitlement, normally Free.

Question 5: How will you demonstrate this module?

I will open Billing, show the current plan and limits, display the checkout or portal controls, show invoice information, and demonstrate that the backend prevents an action beyond the allowed entitlement.

The exact answer style for the examiner

For every module, answer in this order:

> “This module is responsible for…”
>
> “It works by…”
>
> “The main files are…”
>
> “I will demonstrate it by…”
>
> “If it fails, the system shows…”

Example:

> “The rollback module is responsible for restoring a previous stable release. It works by selecting the stored immutable task definition, validating its runtime configuration, updating ECS, waiting for stability, and verifying the public endpoint. The main files are orchestration.service.ts, ecs.service.ts, and the stable-release entity. I will demonstrate it by deploying two versions and rolling back to the first one. If it fails, DeployGuard records ECS events, task status, target health, and a deterministic failure code.”

CURRENT HEAD

a571c14e9afc45bedf22ec1e4d22e136e8e3005e on test2.

PHASE

Viva preparation organized into working explanation, files, demo, and five questions per module.

SCOPE

All 17 modules from the PDF.

FILES CHANGED

None.

CONTRACT BEFORE

No repository files were changed.

CONTRACT AFTER

No repository files were changed.

TEST RESULTS

Local implementation tests previously passed for authentication, build-target detection, deployment requirements, low-risk defect closure, Railpack runtime, Trivy, notifications, lifecycle, Terraform export, direct ECS redeploy, direct ECS rollback, and general billing logic.

AWS E2E STATUS

UNVERIFIED.

UNVERIFIED

Exact AWS deployment, live ECS behavior, backup restoration, external notification delivery, live AI provider, and Stripe E2E.